

Dimensionality Reduction

Overview

23 Mar 2026

- **feature vectors**
- **data manifold**
- **how Principal Component Analysis finds a linear subspace**
- **applications in images and data visualization**

Reading

Pattern Recognition and Machine Learning

12.1: Principal Component Analysis (PCA)

Principal Component Analysis

Core Idea

- **PCA** is a method for **finding a lower-dimensional** representation of data that **preserves** as much of the **important structure** as possible
- **PCA looks** for a small number of **new axes**, called **principal components**, such that:
 - the projected **data varies** as **much as possible** along those axes
 - the **axes** are mutually **orthogonal**
 - the **lower-dimensional** representation **loses** as **little information** as possible **under a squared-error** notion of **reconstruction**

Why PCA is Useful

- **real data** often **lives** in a **high-dimensional** space
- the **true degrees of freedom** may be **much smaller**
- **PCA helps** by:
 - **reducing dimensionality**
 - **compressing** data

	○ removing redundancy caused by correlated variables ○ making visualization easier ○ preparing data for later modeling steps
Setup	• suppose each data point is a vector $x_n \in R^D$ • PCA first centers the data by subtracting the sample mean, so the transformed data has mean 0 • then, studies the sample covariance matrix, whose eigenvectors determine the principal directions • in Bishop's development, the principal components are obtained from the eigenvalue problem for the covariance matrix
View 1: Maximum-Variance Formulation	
Idea	• the first principal component is the direction onto which the projected data has the largest possible variance • Then: ○ the second principal component is the direction of largest remaining variance ○ it must be orthogonal to the first ○ the third is orthogonal to the first two ○ and so on
Key Result	• the optimal directions are the eigenvectors of the covariance matrix ○ the eigenvector with the largest eigenvalue gives the first principal component

	○ the next largest eigenvalue gives the second component ○ larger eigenvalue = more variance explained by that component
View 2: Minimum-Error Formulation	
Idea	• PCA can also be derived as a reconstruction problem • project the data into a lower-dimensional subspace • then reconstruct it back into the original space • choose the subspace that minimizes the average squared reconstruction error
Important Conclusion	• the solution to this minimum-error problem is the same subspace found by the maximum-variance approach • so PCA has two equivalent interpretations: ○ keep directions of largest variance ○ minimize information loss under squared reconstruction error
Reconstruction picture	• if you keep only $M < D$ principal components: ○ each point is projected onto an M – <i>dimensional</i> linear subspace ○ the reconstruction is the closest point to the original data point within that subspace ○ the discarded components correspond to lost information
Choosing Fewer Dimensions	• if you keep only the first few principal components, you get a compressed representation • this works well when:

	○ the first few eigenvalues are much larger than the rest ○ most variance is concentrated in a small number of directions • in that case, the data can be approximated well using a low-dimensional linear manifold
High-Dimensional Data Case	
Large Dimensionality Case	• directly solving the covariance eigenvalue problem can be expensive • a useful trick is to work with a matrix involving inner products between data points instead • this is especially helpful when the number of data points is smaller than the ambient dimension • the nonzero eigenvalues are shared across these equivalent formulations • this matters because PCA is often applied in settings like: ○ images ○ text features ○ other very high-dimensional observations

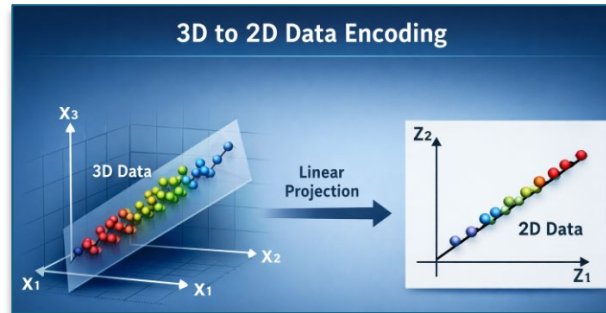
What PCA is Actually Finding	
PCA Finds a Linear Subspace	• assumes the important structure can be captured by straight-line combinations of basis directions • does not model nonlinear curved manifolds • later methods in the chapter extend this idea probabilistically and nonlinearly
Limits of PCA	• it is linear • it is driven by variance, not necessarily by task relevance • large variance directions are not always the most meaningful for prediction • it is sensitive to variable scaling unless preprocessing is done carefully • standard PCA is not a full probabilistic model in this initial formulation
Summary	
PCA Section 12.1	• can be understood as: ○ a way to find a low-dimensional representation of high-dimensional data ○ a method based on eigenvectors of the covariance matrix ○ a maximum-variance method ○ equivalently, a minimum squared reconstruction error method ○ a linear dimensionality reduction technique that prepares the ground for probabilistic PCA and related latent-variable models

Dimensionality Reduction		
Another Type of Unsupervised Learning Task		
Supervised vs Unsupervised Learning	Supervised Learning - data: (x, y) x is data y is label - goal: - learn a function to map $x \rightarrow y$ - examples: classification - regression object detection - semantic segmentation image captioning	Unsupervised Learning - data: x - just data no labels - goal: learn underlying hidden structure of the data - examples: clustering - dimensionality reduction feature learning - density estimation
Dimensionality Reduction	unsupervised learning method - discovering the degrees of freedom of the underlying instances - what are the core dimensions of the dataset	

Linear Encoder and Decoder

Encode

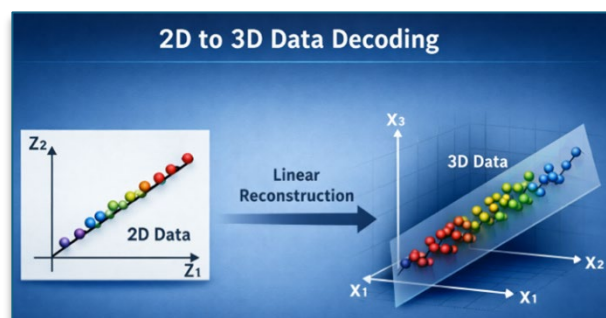
- for 3-D data, **encoding** converts the **3-D data to 2-D**

*ChatGPT 5.4*

- data preparation (**normalization**)
- **compute the covariance matrix**
- apply **SVD**
- **project the 3-D vertices onto the plane formed by the first two eigenvectors**

Decode

- for 3-D data, **decoding** converts the **2-D data back to 3-D**

*ChatGPT 5.4*

- **map the 2D points back to the 3D space using the transpose of the principal components matrix**

- add the **mean** (gravity center) of the **original data** back to the **reconstructed points** to place the **object** in its **original** position
- if the **2D representation** was **heavily compressed**, **surface refinement** or **3D convolutional neural networks**

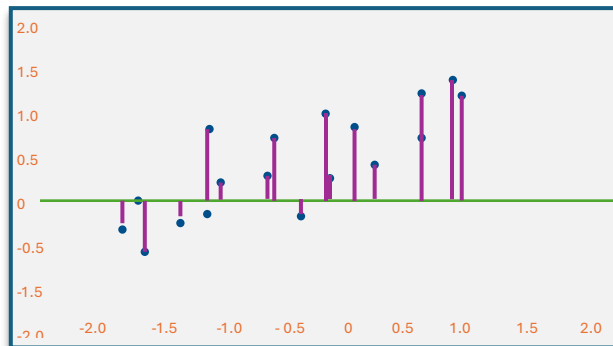
Linear Encoder and Decoder

	encode	decode
K-Means	• $x \in \mathbb{R}^d \rightarrow \{1, \dots, k\}$	• $\{1, \dots, k\} \rightarrow CCenter(x) \in \mathbb{R}^d$
Dim-Reduction	• $x \in \mathbb{R}^d \rightarrow \mathbb{R}^k, k < d$	• $z \in \mathbb{R}^k \rightarrow \hat{x} \in \mathbb{R}^d$

Two Views on Dimensionality Reduction

Simple Dimensionality Reduction

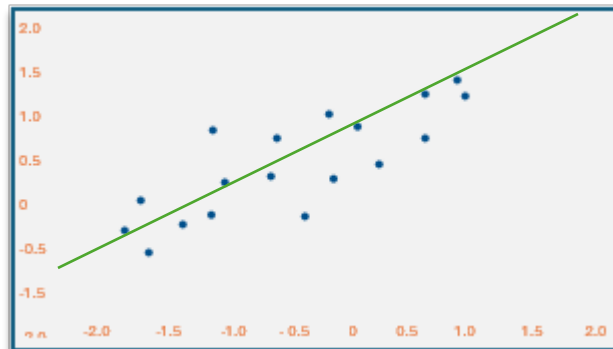
- using a **2-D scatter plot** as an example, one “**view**” would be to **project** the set **directly** to the **x-axis**



PCA

- this method has us **determining** a linear **frame** of reference **corresponding to** the direction of **greatest variance**

the projection here is more “spread out” giving a broader representation of the data



Maximum Variance View

PCA: Max
Variance

- PCA is a **linear dimensionality reduction** technique
- find the **directions** of **maximum variance** in the data

$$\langle \langle x_i \rangle \rangle_{i=1}^N$$

- **assume** that the **data** is **centered**, i.e.

$$\sum_i x_i = 0$$

To Center Your
Data at $\mu = 0$

- **centered data** transformation

$$x'_i = x_i - \mu, \quad \text{where} \quad \mu = \left(\frac{1}{N}\right) \sum_{i=1}^N x_i$$

to center data at zero:

- find μ (mean of data set)
- subtract μ from each datapoint
(linear transformation)

- thus,

$$x'_i = x_i - \left(\frac{1}{N}\right) \sum_{i=1}^N x_i$$

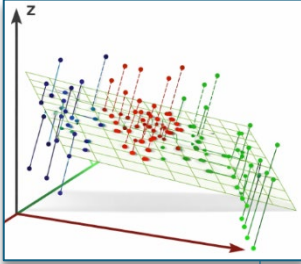
PCA: Max
Variance, cont.

- **find** a set of **orthogonal** vectors $v_1, \dots, v_k$
 - the **first** principal component (PC) v_1 is the **direction** of largest **variance**
 - the **second** PC v_2 is the **direction** of largest **variance** **orthogonal** to v_1
 - the i^{th} PC v_i is the **direction** of largest **variance** **orthogonal** to $v_1, \dots, v_{i-1}$

- $V_{D \times k}$ gives projection

$$z_i = V^T x_i \quad \text{for datapoint } x_i$$

$$Z = XV \quad \text{for entire dataset}$$

Expressed in Matrix Notation	$V = \begin{bmatrix} & & & \\ v_1 & v_2 & \dots & v_k \\ & & & \end{bmatrix} \text{ where, } x_i \in \mathbb{R}^D$ $z_i = v^T x = \begin{bmatrix} -v_1- \\ -\dots- \\ -v_k- \end{bmatrix} \begin{bmatrix} \\ x_i \\ \end{bmatrix} = \begin{bmatrix} v_1^T x_i \\ \dots \\ v_k^T x_i \end{bmatrix} \text{ where, } z_i \in \mathbb{R}^k$
PCA: Max Variance, cont.	- we wish to find v_1 so that $\sum_{i=1}^N z_i^2$ is maximized $\sum_{i=1}^N z_i^2 = z^T z = v_1^T X^T X v_1$ - the maximum value attained by $v_1^T X^T X v_1$ for $\ v_1\ = 1$ important constraint - is the largest eigenvalue of $X^T X$
Reconstruction View	
Best Reconstruction	- we have i.i.d. data $\langle x_i \rangle_{i=1}^N$ data matrix X - find a k-dimensional linear projection that best represents the data - suppose $V_k \in \mathbb{R}^{D \times k}$ is such that columns of V_k are orthonormal - project data X onto subspace defined by V_k - minimize reconstruction error $\sum_{i=1}^N \ x_i - \underbrace{V_k V_k^T x_i}_{\substack{\text{reconstructed} \\ \text{datapoint} \\ z_i}}\ ^2$ initial datapoint 

Singular Value Decomposition

Full vs Thin SVD

$$X = \underbrace{U \Sigma V^T}_{\text{SVD}} \quad (\text{assume } N > D)$$

- thin SVD

$$U \text{ is } N \times D,$$

$$\Sigma \text{ is } D \times D,$$

$$V \text{ is } D \times D,$$

$$U^T U = V^T V = I_D$$

- Σ is diagonal with $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_D \geq 0$
- The first k principal components are the first k columns of V

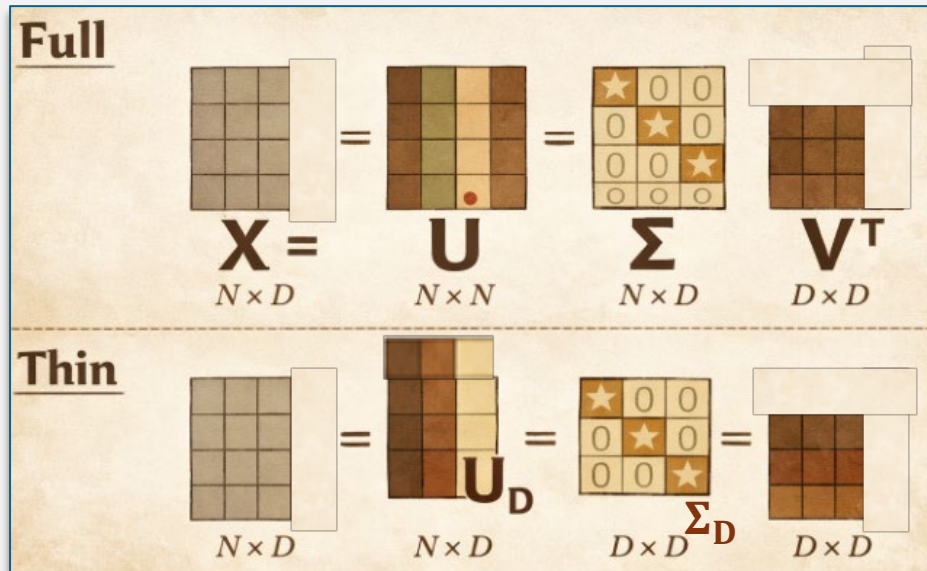
- Full SVD:

$$U \text{ is } N \times N,$$

$$\Sigma \text{ is } N \times D,$$

$$V \text{ is } D \times D$$

U and V are
orthonormal
matrices

Full vs Thin SVD
Visualization

Selecting K **How Many
Principal
Components to
Select**

- look for the “**elbow**” in the **curve of reconstructed error vs number of principal components**

